

SOMIOD: Middleware Orientado a Serviços para Interoperabilidade IoT

Caso de Estudo: Sistema de Gestão Prisional Inteligente

Este projeto foca-se no SOMIOD, um middleware criado para facilitar a troca de dados entre diferentes sistemas. Ele funciona através de uma API RESTful onde podemos criar e gerir recursos de forma organizada (Application, Containers, Content-Instances e Subscriptions). Todos os dados são guardados numa base de dados SQL Server, e o sistema usa o protocolo MQTT para enviar notificações instantâneas sempre que algo acontece.

O sistema foi validado através de um cenário de Gestão Prisional Inteligente. Usando um Monitor e um Controlador, provámos que o middleware consegue ligar aplicações e gerir recursos de forma eficaz.

Keywords—*IoT, Middleware, REST API, MQTT, Interoperabilidade, C#.*

I. INTRODUCTION

Atualmente, vivemos rodeados de dispositivos inteligentes, desde sensores de luz a sistemas de segurança. No entanto, o grande problema é que estes aparelhos são fabricados por empresas diferentes e, por norma, não falam a mesma língua. Isto cria sistemas "fechados", onde os dados de um sensor não conseguem ser lidos por uma aplicação de outra marca. O objetivo principal deste trabalho é quebrar essas barreiras através da criação do SOMIOD, um sistema que funciona como uma ponte (middleware). Este sistema permite que diferentes aplicações troquem informações e recebam alertas de forma simples e organizada, independentemente de como foram programadas.

A nossa solução foca-se em dois pontos principais: a organização dos dados, para que tudo fique guardado no sítio certo, e a rapidez dos avisos, para que os utilizadores saibam o que está a acontecer no momento exato.

Para mostrar que o sistema funciona mesmo na vida real, decidimos criar um cenário de um Sistema Prisional Inteligente. Escolhemos este exemplo porque uma prisão é um local onde nada pode falhar e a comunicação tem de ser instantânea:

- **Monitor (Aplicação A):** É a aplicação que monitora os estados das celas, ela recebe as notificações através do MQTT que envia a informação das celas.
- **Controlador (Aplicação B):** É a aplicação responsável pelo controlo da prisão, nela é possível controlar as celas, esta comunica com a base de dados que depois é lida pelo mosquito que envia o estado para o monitor.

O documento está organizado da seguinte forma:

A **Secção II** mostra o desenho geral do sistema e como as peças se ligam;

A **Secção III** explica a parte técnica e como as bases de dados foram montadas;

A **Secção IV** detalha como criámos as aplicações da prisão para os testes;

A **Secção V** faz um resumo do que correu bem e apresenta as conclusões finais.

II. SYSTEM ARCHITECTURE

O sistema segue uma arquitetura orientada a serviços e recursos, conforme ilustrado:

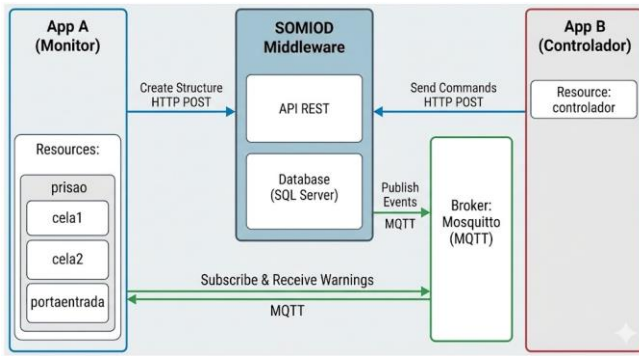


Ilustração Figura 1

A. Componentes Principais

O sistema é dividido em três partes principais que trabalham juntas para que tudo funcione. O coração do projeto é o Middleware SOMIOD.

SOMIOD Middleware: Este é o núcleo do sistema, construído com a tecnologia **ASP.NET Web API**.

Ele funciona como um "recepcionista" que:

- **Cria os caminhos (endpoints):** Define como as outras aplicações podem pedir ou enviar dados.
- **Gere as regras:** Decide como a informação deve ser organizada e guardada.
- **Faz a ponte:** Garante que, quando alguém envia uma informação, ela chega corretamente à base de dados ou avisa quem estiver à espera de alertas.

Base de Dados (SQL Server): Armazena a hierarquia dos recursos (Application, Container, Subscription, Content-Instance). Garante a persistência do estado do sistema prisional.

Message Broker (Mosquitto): Responsável por encaminhar as notificações assíncronas via protocolo MQTT quando novos dados são inseridos.

B. Hierarquia de Recursos

Para manter os dados organizados, a base de dados segue uma estrutura em árvore. Isto permite que as aplicações saibam exatamente onde colocar ou procurar informação:

- **Application (Raiz):** O nível mais alto da estrutura (ex: "Prisão" ou "Controlador").
- **Container (Agrupador):** Funciona como uma pasta para organizar os temas (ex: "Cela 1" ou "Geral").
- **Content-Instance (Dados):** São as informações reais enviadas, muitas vezes em formato XML (ex: ABRIR).
- **Subscription (Notificações):** Guarda a configuração de quem deve ser avisado e para onde deve ser enviada a mensagem:
- **MQTT:** Para avisos instantâneos via broker. (ex: mqtt://127.0.0.1:1883).
- **HTTP:** Para enviar um aviso diretamente para um servidor web ou outra aplicação.

(ex: http://127.0.0.1:5000/api/somiod/notification).

III. EVALUATION

A. RESTful API & Routing

A API funciona como o ponto de entrada do sistema e foi organizada em Controladores específicos (*ApplicationController*, *ContainerController*, *ContentInstanceController* e *SubscriptionController*). Estes controladores recebem e respondem a pedidos usando os métodos padrão da internet: **GET**, **POST**, **PUT** e **DELETE**.

Caminhos das Rotas (Routing): Para que cada recurso tenha o seu lugar, os endereços (URLs) seguem o padrão hierárquico exigido:

- **Aplicações:**
/api/somiod/{appName}.
- **Contentores:**
/api/somiod/{appName}/{contName}.
- **Dados (Content-Instances):**
/api/somiod/{appName}/{contName}/{ciName}.
- **Avisos (Subscriptions):**
/api/somiod/{appName}/{contName}/subs/{subName} — onde usamos a palavra **"subs"** para o sistema não confundir avisos com dados.

Lógica de Duplicação e Integridade: Uma decisão importante na programação foi a forma como lidamos com recursos que já existem. Para evitar erros e manter o sistema estável:

- Ao criar uma Aplicação com um *resource-name* já utilizado em outra, esta aplicação é criada utilizando o seguinte formato de nome "*resource-name_<id_na_base_de_dados>*".
- Isto garante a integridade do cenário da "Prisão": se o sistema for reiniciado, o Monitor e o Controlador continuam a usar os mesmos recursos sem perder as ligações.

B. Persistência de Dados

(BD_Access) Para guardar toda a informação de forma permanente, o middleware liga-se a uma base de dados SQL Server. Esta ligação é feita através de uma tecnologia chamada ADO.NET (SqlClient), que permite ao código falar diretamente com a base de dados para ler e escrever dados.

Para garantir que a informação está sempre correta e organizada, utilizamos regras:

- **Organização Hierárquica:** A base de dados obriga a que a estrutura em árvore (Aplicação -> Contentor -> Recurso) seja respeitada em todos os momentos.
- **Registo de Tempo:** Cada vez que algo é criado, o sistema guarda automaticamente o momento exato (data e hora), conforme exigido pelas regras do projeto.

C. Notificações e Eventos (MQTT e HTTP)

O sistema não se limita a guardar dados; ele também avisa as aplicações sempre que algo novo acontece. Esta funcionalidade é gerida da seguinte forma:

Gatilho de Alerta: Sempre que um novo dado (*content-instance*) é criado com sucesso, o Middleware verifica se existem subscrições ativas para aquele contentor.

Envio de Mensagens: Se existirem subscrições, o sistema utiliza a biblioteca M2Mqtt para publicar a informação no tópico configurado (Mosquitto) ou faz um envio direto via HTTP para o endereço registado.

Conteúdo do Aviso: A notificação enviada inclui o dado que foi criado, para que a aplicação que recebe o aviso (Monitor da prisão) saiba exatamente o que mudou sem ter de perguntar ao servidor.

Suporte a Notificações via Protocolo HTTP Para garantir a interoperabilidade do Middleware SOMIOD e cumprir o requisito de independência de protocolos de transporte, o recurso Subscription foi implementado com suporte híbrido para terminais de notificação (endpoints). Além do suporte nativo ao protocolo MQTT, o sistema permite o registo de endereços baseados no protocolo HTTP/HTTPS, funcionando como um mecanismo de Webhook.

1. Validação e Registo do Endpoint

Durante a operação de criação de uma subscrição (POST), o sistema aceita no campo endpoint URIs que sigam o esquema `http://` ou `https://`. Ao contrário dos endpoints MQTT (que requerem um broker), estes endereços representam serviços RESTful externos que estão à escuta de eventos. O Middleware armazena esta informação na base de dados de forma agnóstica, preservando o formato original do endereço fornecido pelo cliente.

2. Lógica de Disparo de Eventos (Dispatch) No momento em que um evento é despoletado (criação ou eliminação de um Content-Instance), o controlador responsável executa uma rotina de verificação do protocolo do endpoint subscrito.

Identificação do Protocolo: O sistema analisa o prefixo do URI armazenado. Se este iniciar por `http` ou `https`, o fluxo de execução é desviado para o módulo de tratamento HTTP.

Construção do Pedido (Request): É instanciado um cliente HTTP (utilizando a classe `HttpClient` do .NET) para construir um pedido com o verbo POST.

Payload da Mensagem: O corpo do pedido HTTP é preenchido com a mensagem de notificação em formato XML, contendo as tags `<event>` e `<resource>`, garantindo a conformidade com o esquema XSD de validação.

Envio Assíncrono: O envio da notificação é realizado de forma assíncrona (*fire-and-forget*). Esta abordagem arquitetural é crítica para assegurar que a latência ou indisponibilidade do serviço recetor não bloqueia o desempenho do Middleware nem atrasa a resposta ao cliente que originou o evento.

Esta implementação permite que aplicações de terceiros (como o Monitor ou outros serviços Web) recebam notificações em tempo real através de portas HTTP padrão (80/443), sem necessidade de bibliotecas de subscrição MQTT, aumentando assim a flexibilidade de integração do sistema.

D. Discovery

Para que as aplicações não tenham de saber todos os nomes de cor, o sistema inclui uma funcionalidade Discovery.

Como funciona: Utilizamos o método GET juntamente com um cabeçalho especial chamado *somiod-discovery*.

O que faz: Se uma aplicação enviar um pedido com *somiod-discovery*: container, o middleware percorre a árvore e devolve uma lista com os caminhos de todos os contentores que encontrar.

Isto facilita muito a navegação, pois permite que o Controlador saiba, por exemplo, quantas celas existem na prisão sem que o programador tenha de escrever esses nomes no código.

E. Documentação com Swagger

Para garantir que o middleware cumpre os padrões de interoperabilidade exigidos e para facilitar a sua utilização por programadores externos, foi integrada a ferramenta Swagger no projeto. A implementação desta documentação é gerada de forma automática recorrendo à biblioteca *Swashbuckle*, que efetua o mapeamento dos controladores e dos respetivos métodos HTTP, nomeadamente GET, POST, PUT e DELETE.

Através do *endpoint /swagger*, o sistema disponibiliza uma interface gráfica interativa que permite a exploração detalhada de recursos, possibilitando a visualização da hierarquia de rotas e dos parâmetros aceites em cada operação. Adicionalmente, o Swagger possibilita a realização de testes em tempo real, permitindo executar pedidos diretamente no navegador para validar o

comportamento da API e os códigos de resposta HTTP sem a necessidade de ferramentas externas. Esta implementação assegura que o middleware se encontra documentado, suportando o desenvolvimento de novas aplicações de forma consistente.

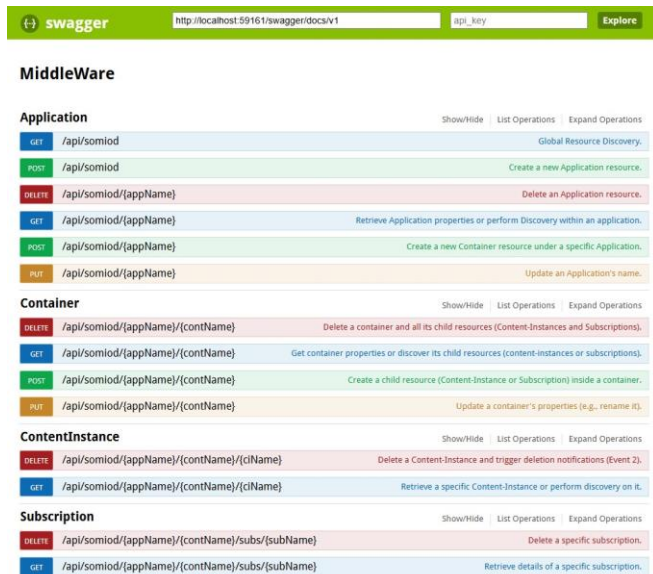


Figura 2: Interface Swagger para documentação dos endpoints do SOMIOD

IV. INTEGRATION/APP DEVELOPMENT

Para validar o middleware, criámos um cenário de **Gestão Prisional Inteligente**. Este cenário demonstra como diferentes aplicações podem comunicar através do SOMIOD para controlar infraestruturas críticas.

Aplicação A: Monitor de Prisão

Esta aplicação simula o ambiente físico da prisão, como as celas e os sistemas de acesso.

1. Inicialização e Configuração: Ao ser ligada, a aplicação comunica automaticamente com a API para preparar a comunicação.

- **Criação da Estrutura:** Garante que a aplicação "prisão" e os contentores "cela1", "cela2" e "porta" existam no sistema.
- **Uso de IDs:** Se estes recursos já existirem, a aplicação reutiliza os IDs atuais, mantendo a continuidade do sistema.
- **Subscrições:** Cria as subscrições necessárias (via MQTT ou HTTP) para começar a ouvir comandos.

2. Funcionamento Real: O Monitor funciona como um receptor de ordens.

- **Escuta Ativa:** Subscrive os tópicos no MQTT para receber notificações instantâneas.
- **Controlo Visual:** Quando recebe um comando (Content-Instance) com o texto "ABRIR" ou

"FECHAR", a interface muda de cor (Verde para aberto, Vermelho para fechado).

- **Modo Lockdown:** Permite fechar todas as celas ao mesmo tempo, garantindo a segurança máxima em caso de emergência.



Figura 3: Monitor Prisão

3. Mecanismo de Validação e Persistência de Notificações (XML/XSD)

Para dar cumprimento ao requisito adicional do enunciado, foi implementado um módulo de segurança e persistência na aplicação de monitorização, garantindo que as notificações recebidas via broker (MQTT) ou Webhook (HTTP) são fidedignas e estão bem estruturadas.

a. Definição do XML Schema (XSD): Foi desenvolvido um ficheiro de esquema (notification.xsd) que define a gramática das mensagens. O esquema utiliza tipos complexos (xs:complexType) e indicadores de sequência (xs:sequence) para obrigar a que cada notificação contenha exatamente os elementos <event> e <resource>, garantindo a integridade dos dados antes do processamento.

b. Processo de Validação Programática: A validação não é apenas visual, mas sim programática, recorrendo às bibliotecas do .NET Framework:

Configuração de Leitura: Utilizou-se a classe XmlReaderSettings com o ValidationType configurado para Schema. **Interceção de Erros:** Foi implementado um ValidationEventHandler (através do método ValidationCallback), que atua como um "sentinela", capturando qualquer desvio às regras do XSD.

Validação em Tempo de Execução: O conteúdo da mensagem é processado por um XmlReader. Se a estrutura for inválida, o sistema bloqueia a execução do comando e descarta a mensagem.

c. Serialização e Persistência: Uma vez validada com sucesso, a notificação é serializada (convertida de fluxo de dados para ficheiro físico). Os ficheiros são guardados localmente com um selo temporal (timestamp) único. Este processo assegura a rastreabilidade de todos os eventos (criação ou eliminação) que ocorrem no Middleware.

d. Segurança e Fiabilidade: Este fluxo garante que a aplicação "Monitor Prisão" apenas reage a comandos que seguem estritamente o protocolo estabelecido, prevenindo

erros de execução por mensagens mal formadas ou incompletas.

Aplicação B: Controlador de Prisão

Esta aplicação funciona como o comando remoto nas mãos dos guardas, permitindo controlar a prisão à distância. Ao contrário do Monitor, esta aplicação não precisa de estar sempre à escuta, agindo apenas quando o utilizador carrega num botão.

1. Registo e Identificação

- **Identidade:** A aplicação regista-se no middleware com o nome "controlador", criando o seu próprio recurso para ser reconhecida pelo sistema.
- **Modo de Operação:** Funciona puramente como um cliente REST. Isto significa que não utiliza o protocolo MQTT, comunicando apenas através de pedidos Web (HTTP) para enviar ordens.

2. Funcionalidades de Controlo: O Controlador apresenta uma interface com vários botões para gerir a segurança da prisão.

- **Comandos Individuais:** Permite escolher uma porta específica (ex: "cela1") e enviar um pedido **HTTP POST** para o respetivo contentor.
- **Ações em Massa:** Inclui a opção de abrir ou fechar todas as portas de uma só vez, enviando comandos para o contentor geral.
- **Formato das Ordens:** As instruções são enviadas como instâncias de conteúdo e o sistema suporta os três formatos: XML, JSON ou texto simples. Isto garante que o Monitor consiga ler e perceber exatamente o que deve fazer, independentemente do formato escolhido para enviar o comando.



Figura 4: Controlador Prisão

V. CONCLUSIONS AND FUTURE WORK

O projeto atingiu todos os objetivos propostos, demonstrando que o middleware SOMIOD é uma solução eficaz para a interoperabilidade entre diferentes sistemas.

Cumprimento de Requisitos: Foram implementadas com sucesso todas as funcionalidades de CRUD, a descoberta de recursos, e o sistema de notificações via MQTT.

Cenário Original: O uso do sistema prisional permitiu validar o middleware num contexto prático e distinto dos exemplos base, garantindo a originalidade do trabalho.

Arquitetura: A estrutura desacoplada provou ser uma vantagem, permitindo que o Monitor (parte física) e o Controlador (parte lógica) evoluam de forma independente sem comprometer o sistema central.

Trabalho Futuro: Como trabalho futuro, sugere-se a implementação de mecanismos de autenticação na API para garantir que apenas utilizadores autorizados possam gerir os recursos. Além disso, seria relevante expandir o sistema de forma a integrar funcionalidades de análise de dados, permitindo gerar relatórios sobre o histórico de acessos e eventos registados na prisão.

REFERENCES

- [1] Uso de LLMs das quais: Gemini, Chat gpt 5 e Github copilot
- [2] Uso de recurso disponibilizados no Moodle pelos Docentes

APPENDIX

Appendix A

Postman Collection - IS.postman_collection.json CRUD commands:

POST Creation Application:

```
curl --location 'http://localhost:59161/api/somiod' \
--header 'Content-Type: application/json' \
--data '{"res-type": "application", "resource-name": "prisao"}
```

POST Creation Container:

```
curl --location 'http://localhost:59161/api/somiod/prisao' \
--header 'Content-Type: application/json' \
--data '{"res-type": "container", "resource-name": "cela1"}
```

POST Creation Content-Instance:

```
curl --location
'http://localhost:59161/api/somiod/prisao/cela1' \
--header 'Content-Type: application/json' \
--data '{
  "res-type": "content-instance",
  "resource-name": "comando_abrir_1",
  "content": "FECHAR",
  "content-type": "application/json"
}'
```

POST Creation Subscription:

```
curl --location
'http://localhost:59161/api/somiod/prisao/cela1' \
--header 'Content-Type: application/json' \
--data '{"res-type": "subscription", "resource-name":
"subDel", "evt": 2, "endpoint": "mqtt://127.0.0.1:1883"}
```

GET Aplication:

```
curl --location 'http://localhost:59161/api/somiod/prisao' \
--header 'Content-Type: application/json'
```

GET Container:

```
curl --location
'http://localhost:59161/api/somiod/prisao/cela1' \
--header 'Content-Type: application/json'
```

GET Content-Instance:

```
curl --location
'http://localhost:59161/api/somiod/prisao/cela1/comando_ab
rir_1' \
--header 'Content-Type: application/json'
```

GET Subscription:

```
curl --location
'http://localhost:59161/api/somiod/prisao/cela1/subs/subCela
1' \
--header 'Content-Type: application/json'
```

GET Discovery Aplication:

```
curl --location 'http://localhost:59161/api/somiod' \
--header 'somiod-discovery: application'
```

GET Discovery Container:

```
curl --location 'http://localhost:59161/api/somiod/prisao/' --
header 'somiod-discovery: container'
```

GET Discovery Content-Instance:

```
curl --location
'http://localhost:59161/api/somiod/prisao/cela1' \
--header 'somiod-discovery: content-instance'
```

GET Discovery Subscription:

```
curl --location
'http://localhost:59161/api/somiod/prisao/cela1' \
--header 'somiod-discovery: subscription'
```

PUT Update Aplication:

```
curl --location --request PUT
'http://localhost:59161/api/somiod/prisao' \
--header 'Content-Type: application/json' \
--data '{"resource-name": "prisao1"}'
```

PUT Update Container:

```
curl --location --request PUT
'http://localhost:59161/api/somiod/prisao/cela1' \
--header 'Content-Type: application/json' \
--data '{"resource-name": "celaA"}'
```

DELETE Content-Instance:

```
curl --location --request DELETE
'http://localhost:59161/api/somiod/prisao/cela1/comando_ab
rir_1'
```

DELETE Subscription:

```
curl --location --request DELETE
'http://localhost:59161/api/somiod/prisao/cela1/subs/subCela
1'
```

DELETE Container:

```
curl --location --request DELETE
'http://localhost:59161/api/somiod/prisao/cela1'
```

DELETE Aplication:

```
curl --location --request DELETE
'http://localhost:59161/api/somiod/prisao' POST
Notification:
```

```
curl --location 'http://localhost:5000/api/notificacao/' \
--header 'Content-Type: application/xml' \
--data '<notification>
  <event>CREATE</event>
  <resource>FECHAR</resource>
</notification>'
```

Appendix B

Francisco - Implementação do controlador de Container e dos modelos de dados (Application, Container, Content-Instance e Subscription). Desenvolvimento da camada de comunicação entre BD e API. Correção da lógica de *Somiod Discovery*, sistema de notificações por HTTP, através de dados em XML e com validação XSD, documentação Swagger e validação funcional via Postman.

Martim - Aplicação da prisão e controlador, implementação e correção dos Controllers do Container, Content-Instance e implementação do broker Mosquito. Implementação do acesso à base de dados. Testes da API com postman para testar broker MQTT ao middleware

Fábio - BD. Implementação e correção do Controller do Container, Content-Instance e do broker Mosquito. Testes da API com postman para testar broker MQTT ao middleware

Bernardo - Criação dos Endpoints, Testes da API com postman, correção de conteúdo na BD, verificação da API e funcionamento geral das Apps.

Cada elemento do grupo começou por desenvolver uma parte específica do projeto, mas a solução final foi trabalhada por todos em conjunto. Depois das implementações iniciais, todos colaboraram na correção da lógica, na resolução de erros e na integração dos vários módulos. Desta forma, garantimos que todos os membros do grupo dominam o funcionamento total do sistema, desde a base de dados até às aplicações de monitorização e controlo.

Para colocar o ecossistema SOMIOD em funcionamento, devem ser seguidos estes passos fundamentais:

1. **Abertura e Compilação:** Abra a solução no **Visual Studio 2022** e execute o comando Build > Build Solution. Este processo gera automaticamente os ficheiros executáveis das aplicações de teste nas respetivas pastas /bin/Debug de cada projeto.
2. **Execução da API e Aplicações:** Clique no botão **Start** (ou prima F5). O Visual Studio irá iniciar simultaneamente:
 - O servidor **IIS Express** para a API SOMIOD.
 - A interface gráfica do **monitor** da prisão. ● A interface de **controlo** da Prisão.